

**Journal 08: The Role of Hyperparameters in AI Model Performance and Ethics**

Sharie Rhea

Southern New Hampshire University

CS 530: Artificial Intelligence Principles and Applications

Joshua Andrews

March 29, 2026

With the increasing popularity of artificial intelligence (AI), understanding how to achieve the best model has become a widely researched challenge. Beyond the design of the neural network itself, hyperparameters—or adjustable settings—are crucial to optimizing performance. First, the two major types of hyperparameters are described in the context of AI models. The usage of hyperparameters during training is discussed, and finally the impact of hyperparameter selection on code accuracy and bias is analyzed.

Model parameters, also called weights, are the values learned during training and updated by the optimization function, such as gradient descent. Hyperparameters, on the other hand, are set either before or after the training process. Pre-training hyperparameters include batch size, number of epochs, depth or number of hidden layers, and dropout rate (Saini, 2025). Post-training hyperparameters include temperature, top probability, frequency penalty, and presence penalty (Arora et al., 2024). The process of adjusting these values to obtain the best possible model as a result is known as tuning.

These distinctions are important because each type of hyperparameter influences a different stage of model development. Pre-training hyperparameters shape how the model learns, helping prevent over- or underfitting, improve efficiency, and support generalization. Batch size controls how many samples are computed before performing weight updates. A large batch size results in faster convergence with smoother gradients but may miss out on small variations (Saini, 2025). The number of epochs, or complete passes through the training data, can have a drastic effect on model fit, with too few epochs causing severe underfitting and too many epochs resulting in memorization of the training data (Saini, 2025). The number of hidden layers within a network, also known as its depth, influences model complexity. Training a shallow model is

typically fast and computationally cost efficient but may fail to capture more intricate data patterns (Saini, 2025). The dropout rate is another crucial hyperparameter to tune, as it prevents overreliance on a single neuron by randomly choosing nodes to exclude during each training step (Saini, 2025). Striking the perfect balance between these pre-training hyperparameters determines whether a model generalizes well and accurately captures patterns in the training data or misses intricacies and has simply memorized features of the input.

While pre-training hyperparameters shape how a model learns, post-training hyperparameters determine how that learned knowledge is expressed during output generation. Temperature is the measure of the output's "randomness," or probability to choose a token other than the most likely next output (Arora et al., 2024). The top probability value restricts token selection to those whose cumulative probabilities fall below a specified threshold (Arora et al., 2024). Frequency penalty penalizes tokens based on how often they have already appeared, while presence penalty penalizes tokens simply for appearing at all in the output so far. (Arora et al., 2024).

Hyperparameter tuning can have a major impact on the accuracy of AI-generated code. Arora et al. (2024) showed that tuning was the difference between success and failure in approximately 10% of Python task generation cases. Complex tasks involving recursion or long descriptions were more likely to result in incomplete or incorrect outputs. Petryshyn & Lukoševičius (2024) showed that post-training hyperparameter tuning had a far greater effect on model performance than the number of parameters alone, as Llama was able to outperform GitHub's Copilot with 25 times fewer parameters. They also noted that models tend to perform best when prompted at the same context size that the model was trained on. Current research

suggests that AI is best for explaining code along with generating documentation, boilerplate code, and unit tests (Pandey et al., 2024). Tuning hyperparameters for a specific use case is often the most effective approach and can be achieved in several ways (Malashin et al., 2025). Malashin et al. (2025) and Morte et al. (2025) both proposed tuning algorithms that produce Pareto fronts, showcasing sets of values where the performance of the considered objective cannot be improved without degrading performance in at least one other.

Hyperparameter tuning can also influence bias in model outputs. Since temperature affects the randomness of generated output, a lower temperature will result in more predictable outcomes. This is desirable for coding, as programming languages have a rigid syntax and accepted patterns for achieving certain tasks (Arora et al., 2024). However, this can hinder creative applications and may amplify existing biases or common patterns in generated content. A similar effect can be observed for top probability. Frequency and presence penalties, on the other hand, affect repetitiveness. Low penalties are desirable for programming where variables must be reused, but may also create stilted and uninspired general outcomes. Higher penalties encourage the model to find synonyms and other replacements to introduce variety.

In summary, hyperparameters are adjustable values that influence an AI model's training or output. The quality, accuracy, and biases of AI-generated content are strongly linked to the chosen hyperparameters, especially in code and software development. Therefore, understanding hyperparameters is critical not only for improving model accuracy but also for mitigating unintended bias in AI-generated outputs. Current research focuses on developing efficient methods for tuning both pre- and post-training hyperparameters.

## References

- Arora, C., Sayeed, A. I., Licorish, S., Wang, F., & Treude, C. (2024). Optimizing Large Language Model Hyperparameters for Code Generation. *ArXiv (Cornell University)*. <https://doi.org/10.48550/arxiv.2408.10577>
- Malashin, I., Masich, I., Kurashkin, S., Gantimurov, A., Borodulin, A., Neluyb, V., & Tynchenko, V. (2025). Automated Evolutionary Hyperparameter Tuning for NLP-Based Test Case Generation. *2025 40th IEEE/ACM International Conference on Automated Software Engineering Workshops (ASEW)*, 228–235. <https://doi.org/10.1109/asew67777.2025.00051>
- Morte, F. B., Sousa, P., Gonçalves, G., & Klautau, A. (2025). Efficient LLM Tuning: A Multi-Objective Approach for Code Generation. *2025 Workshop on Communication Networks and Power Systems (WCNPS)*, 1–7. <https://doi.org/10.1109/wcnps69127.2025.11295914>
- Pandey, R., Singh, P., Wei, R., & Shankar, S. (2024). Transforming Software Development: Evaluating the Efficiency and Challenges of GitHub Copilot in Real-World Projects. *ArXiv (Cornell University)*. <https://doi.org/10.48550/arxiv.2406.17910>
- Petryshyn, B., & Lukoševičius, M. (2024). Optimizing Large Language Models for OpenAPI Code Completion. *ArXiv (Cornell University)*. <https://doi.org/10.48550/arxiv.2405.15729>
- Saini, D. (2025, September 22). *Hyperparameters 101: Your One-Stop Guide for All Deep Learning Models (Part 1)*. Medium; Artificial Intelligence in Plain English. <https://ai.plainenglish.io/hyperparameters-101-your-one-stop-guide-for-all-deep-learning-models-part-1-86f163f58a53>